

Yes. This recording is mainly clarifying **customer acceptance** and **the pricing reward**, and it also confirms how the routing probabilities enter the pricing model.

I'll explain it simply.

1. Customer acceptance probability

The first issue is:

Given the price offered to a customer, what is the probability that the customer accepts the ride?

Sir gives us **three possible ways** to model this.

Option A — Use a published paper

Find a paper that gives a customer ride-acceptance probability/function and use that formulation.

This is preferred because we can say:

"The customer acceptance behaviour is modelled using the formulation reported in [paper]."

That is much easier to defend academically than saying we randomly invented the probability.

Option B — Use our historical data

We could analyse the historical data and estimate price sensitivity.

For example:

Price increases
↓
Acceptance probability decreases

Then fit a distribution/function to the data.

Sir mentions using a truncated normal distribution as one possibility.

Option C — Use a price-elasticity function from another paper

The discussion also mentions a paper using price elasticity / a functional relationship.

Which should we use?

Sir's preference is essentially:

Use a published formulation if possible.

The reason is generalizability. If we estimate it only from NYC historical data, someone could ask whether the result applies elsewhere.

So we should wait for the customer-preference paper Sir said he would send before finalizing this module.

2. What exactly do we need from that paper?

This is important.

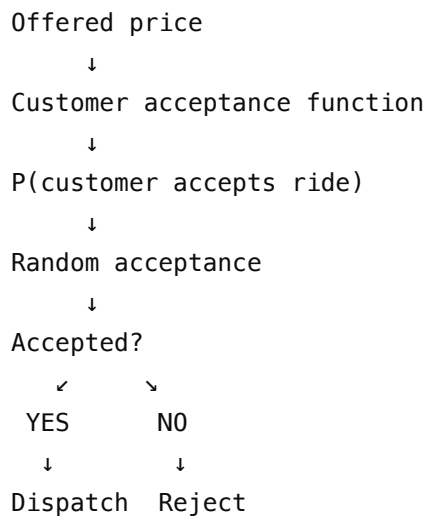
Sir says the paper contains **two probabilities**:

1. probability that a customer chooses ride sharing
2. probability that a customer chooses only ride hailing

We need **only the ride-hailing probability**.

So we should not accidentally implement the ridesharing probability.

Our eventual flow will be:



3. Now the pricing reward is clearer

You asked earlier what the pricing model is trying to maximize.

This recording answers that.

The pricing reward is based on **revenue from accepted/served trips**.

Sir says:

accumulate the total revenue obtained during the slot.

So suppose during a 30-minute slot we have:

Trip 1 → \$15
Trip 2 → \$20
Trip 3 → \$12
Trip 4 → \$18

Then:

$$R_t = 15 + 20 + 12 + 18$$

$$R_t = \$65$$

That becomes the pricing reward for that slot.

4. There is also an average-revenue possibility

The discussion mentions another possibility:

$$R_t = \frac{\text{total accepted-trip revenue}}{\text{number of accepted trips}}$$

The transcript says this could be the average of the total revenue in the slot.

But the discussion ultimately confirms that the objective is based on **accepted-trip revenue**.

For our implementation, I would record:

Baseline pricing reward = accumulated revenue from accepted/served trips during the 30-minute slot.

If we later want average revenue per accepted trip, that should be a separate experiment, not silently substituted into the baseline.

5. Very important: acceptance and dispatch are two different things

There is a slightly confusing sentence in the transcript:

"it does not matter whether customer has accepted it, the dispatch should happen."

The surrounding conversation makes the intended operational flow important.

The key distinction is:

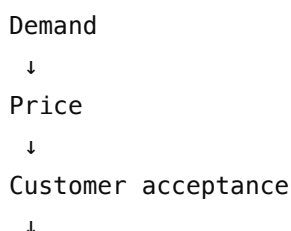
Customer acceptance

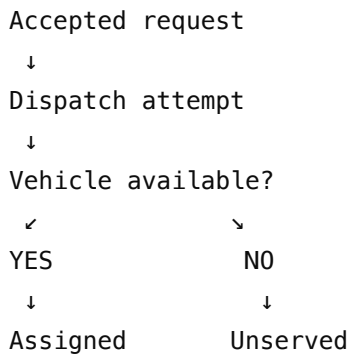
Determines whether the request becomes a **real demand/dispatch candidate**.

Dispatch

Once a dispatch candidate exists, the dispatch engine tries to find a vehicle.

So conceptually:





The previous baseline we built did not have the customer-acceptance stage. That is one of the things we still need to integrate.

6. Routing probabilities enter pricing

This is another major clarification.

The routing policy produces the **complete routing probability vector**.

For example, for Grid 293:

```
STAY    = 0.20
NORTH   = 0.40
EAST    = 0.25
SOUTH   = 0.05
WEST    = 0.10
```

They add up to:

1.00

That **entire vector** becomes part of the pricing model's input.

Sir explicitly says the pricing learner should receive the complete 5-action routing probability vector.

So we should **NOT** give the pricing model only the highest-probability action.

It gets:

$$[P(STAY), P(NORTH), P(EAST), P(SOUTH), P(WEST)]$$

7. Why does each grid have its own routing probability?

Because routing behaviour depends on the state of that particular grid.

For example:

Grid 293

Demand = high
Price = high
Wait = low
Charging = available

Routing probabilities:

STAY 0.20
NORTH 0.40
EAST 0.25
SOUTH 0.05
WEST 0.10

Grid 320

Could have:

Demand = low
Price = low
Wait = high
Charging = unavailable

Routing probabilities:

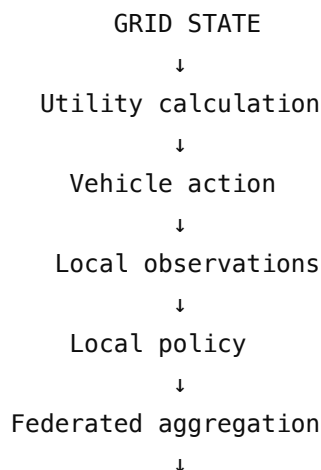
STAY 0.05
NORTH 0.60
EAST 0.20
SOUTH 0.10
WEST 0.05

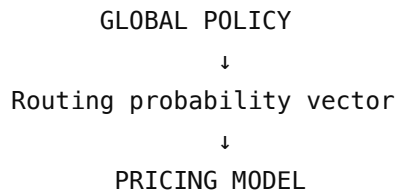
So the pricing model receives **grid-specific routing behaviour**.

The recording explicitly says that for each grid there is its own routing probability based on the forecasted price, waiting time and availability for that 30-minute period.

8. Where does the global routing model fit?

This now gives us a nice architecture:





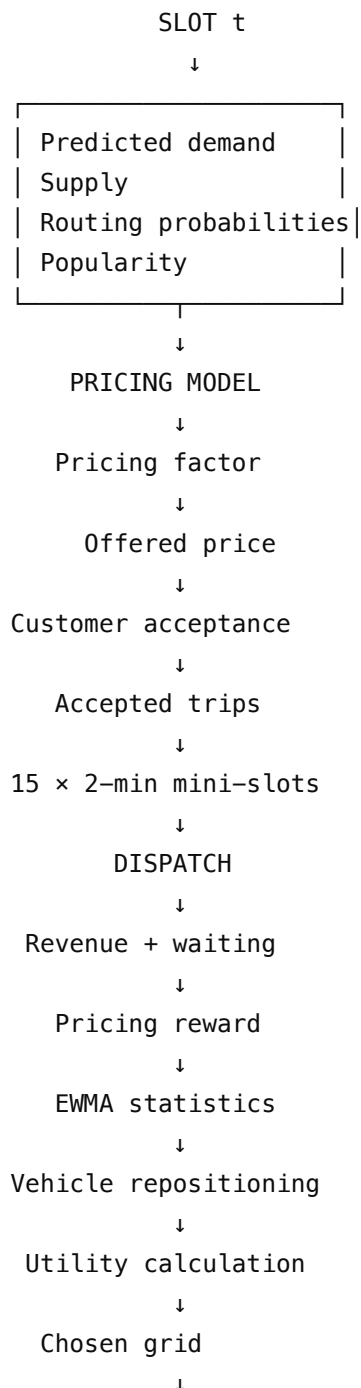
For each grid:

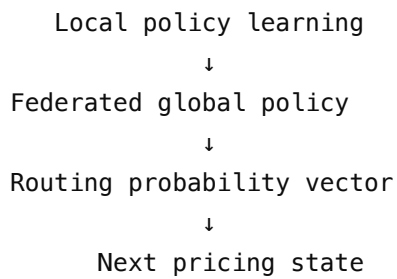
$$\pi_g = [P(STAY), P(NORTH), P(EAST), P(SOUTH), P(WEST)]$$

The pricing model uses that vector as part of its state.

9. Putting everything together

The final 30-minute slot becomes:





10. What is now confirmed vs still pending

Confirmed by this recording

Item	Decision
Pricing reward	Accepted/served-trip revenue
Reward timing	Per 30-minute slot
Revenue handling	Accumulate revenue during slot
Routing input to pricing	Full routing probability vector
Routing vector	All 5 actions
Routing probabilities	Grid-specific
Customer acceptance	Probability-based
Acceptance source	Prefer published paper
Customer paper	Sir will provide
Paper probabilities needed	Ride-hailing probability only

Still pending

1. Customer acceptance equation

We need the paper Sir mentioned.

2. Exact pricing learner

The overall plan allows MAB or DQN. We still need to lock that down.

3. Exact base-price calculation

We know the pricing factor modifies the price, but the final base-fare formulation still needs to be explicitly fixed if it isn't already in the implementation plan.

4. Corrected routing implementation

We now have enough information to correct NB11/NB12:

Utility
↓
Maximum utility
↓
Observed action
↓
Policy network
↓
Routing probabilities

rather than the old DQN routing implementation.

The most important takeaway

Think of the system as **two learning systems**:

Routing learns vehicle behaviour

State → Utility → Vehicle Action → Policy → Routing Probabilities

Pricing learns platform behaviour

Demand + Supply + Routing Probabilities → Pricing Factor → Customer Acceptance

That separation is now quite solid based on the three supervisor discussions we've reviewed. The **one thing I would not implement yet is customer acceptance**, until Sir sends the referenced paper, because he specifically proposed using that published probability formulation rather than us inventing one.